

LINGI2142 : Computer Networks configuration and management

Ceci est une mise au propre de mes notes de cours prises durant le cours LINGI2142 Computer Network and management 2019-2020. Pour des raisons de facilité et de rapidité d'écriture, toutes les explications sont fournies en français.

Auteur : Grégory Creupelandt

Table des matières

1. Multipath-TCP :	3
2. eBGP :	4
3. iBGP :	4
3.1 Principes généraux	4
3.2 Cas full-mesh sans utilisation de Route-Reflector	5
3.3 Cas MPLS	6
3.4 Cas des Route-Reflectors :	6
4. Critères de sélection des routes pour eBGP et iBGP	7
4.1 Ignorer les routes dont le BGP nexthop est injoignable (dead)	7
4.2 Préférer les routes avec une loca-pref élevée	8
4.3 Préférer les routes avec un AS path plus court	9
4.4 Préférer les routes avec un MED plus petit	10
4.5 Préférer les routes apprises par eBGP plutôt que par iBGP	10
4.6 Préférer les routes avec le NextHop le plus proche	10
4.7 Préférer les routes apprises par le routeur eBGP d'ID le plus bas	11
5. Multicast	11
5.1 Principes généraux	11
5.2 Modifications au niveau du Sender	12
5.3 Modifications au niveau du Receiver	12
5.4 IGMP	12
5.5 ASM	12
5.6 SSM	13
5.7 Methodes de construction de tree	13
6. MPLS	15
6.1 Principes généraux :	15
6.2 Distribution des Labels	15
7. Load Balancing	17
8. Failure Recovery	19
8.1 Failure detection	19
8.2 Failure Fast Recovery	20
9. VPN	23
9.1 Principes généraux :	23
9.2 Provider-Provisioned MPLS VPN :	24
10. Segment Routing	27

Network 2 : Notes de cours

Remarque : Je considère que la matière de Network 1 est acquise et ne vais donc pas trop m'attarder dessus ici.

1. Multipath-TCP :

Il s'agit d'une évolution de TCP, fonctionnant sur un network IPV4 ou IPV6, globalement partout où le TCP classique fonctionne. L'idée est d'avoir plusieurs connexions TCP liées entre-elles comme si elles ne formaient qu'une seule connexion, par exemple pour échanger plus rapidement des données.

Une connexion TCP est identifiée par 4 informations :

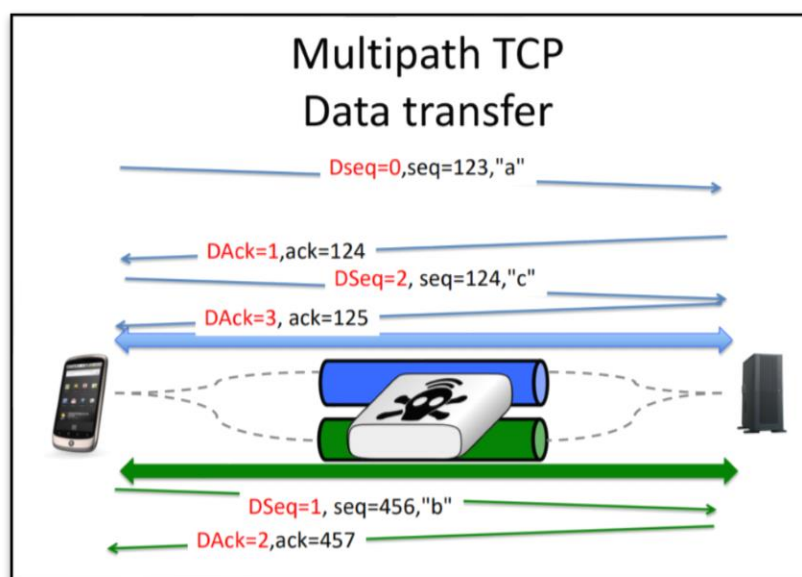
IP source

Port source

IP destination

Port destination

En pratique : pour créer un Multipath-TCP avec deux connexions TCP liées, les deux connexions TCP vont être initialisées (3-way handshake) avec un token en option. Le serveur possède un token et le client possède un token différent. Quand l'un envoie un paquet, il met en option le token de l'autre de façon à lui indiquer de regrouper ces paquets comme étant une unique connexion TCP portant ce nom (nom lié au token). Pour une connexion TCP, les paquets doivent être ordonnés en suivant le sequence-number. Par conséquent, pour permettre aux deux connexions TCP liées de suivre leurs propres seqnumber mais également aux données envoyées d'être ordonnées, deux types de numéros de séquence sont utilisés ; SeqNumber et DataSeqNumber (même principe pour les acknowledgements avec ack et Dack). Pour gérer les pertes, il y a utilisation de fast-retransmit comme pour le TCP régulier et pour la congestion window, les deux connexions TCP liées vont utiliser une window commune.



2. eBGP :

Les sessions eBGP sont des connexions TCP établies entre deux routeurs d'AS différents. Un routeur en session eBGP va advertise sa meilleure route pour chaque préfixe connu pourvu que ce soit autorisé par le filtre d'exportation. Ces filtres sont généralement configurés pour correspondre à un lien client, lien provider, lien shared-cost, lien backup, etc... La configuration des filtres eBGP ainsi que d'autres paramètres tels que la modification des local-pref peut avoir un impact important sur le network. Plusieurs astuces peuvent également être utilisées afin de favoriser un lien eBGP plutôt qu'un autre. Par exemple, il est possible d'advertise un préfixe plus large sur un lien eBGP d'un routeur et un préfixe plus précis depuis un autre routeur eBGP. Les paquets arrivant de l'autre AS préféreront passer par le lien par lequel le préfixe le plus précis a été advertised tout en utilisant l'autre lien en cas de back-up. Mais cela a un impact sur la mémoire des routeurs car ils auront une ligne de plus dans leur table de routing (car préfixe différent) et donc si tout le monde utilise ce type d'astuces, il y a un problème de scaling à très grande échelle. Par exemple, on peut aussi utiliser une local-pref élevée concernant un lien eBGP et une local-pref plus basse sur pour autre lien qui servira de back-up (remarque : une local-pref ne peut être transportée que par un paquet de iBGP pas dans un paquet eBGP, cela concerne donc la configuration interne à un AS) . D'autres astuces existent mais ont généralement un contre-coût. Par exemple, on peut volontairement allonger un AS path sur un des liens eBGP en annonçant AS2,AS2,AS3 au lieu de simplement AS2,AS3 mais cela a encore un impact sur la mémoire des routeurs. Une meilleure solution pour ce cas spécifique est d'utiliser des MED (pas d'impact global). Enfin, les bgp communities sont des flags qui peuvent être ajoutés dans les paquets d'eBGP, par exemple, pour tag une région géographique, le type de lien eBGP (client, provider, shared-cost), demander à l'autre AS d'utiliser le lien d'une façon spécifique, etc... Il est cependant conseillé d'utiliser des communities pour de petits networks/AS mais de ne pas les envoyer vers l'extérieur (Global Internet) pour ne pas polluer le réseau à grande échelle.

3. iBGP :

Comment transmettre ces routes extérieures aux autres routeurs internes :

- iBGP full-mesh
- Uniquement MPLS
- iBGP avec Route-Reflectors

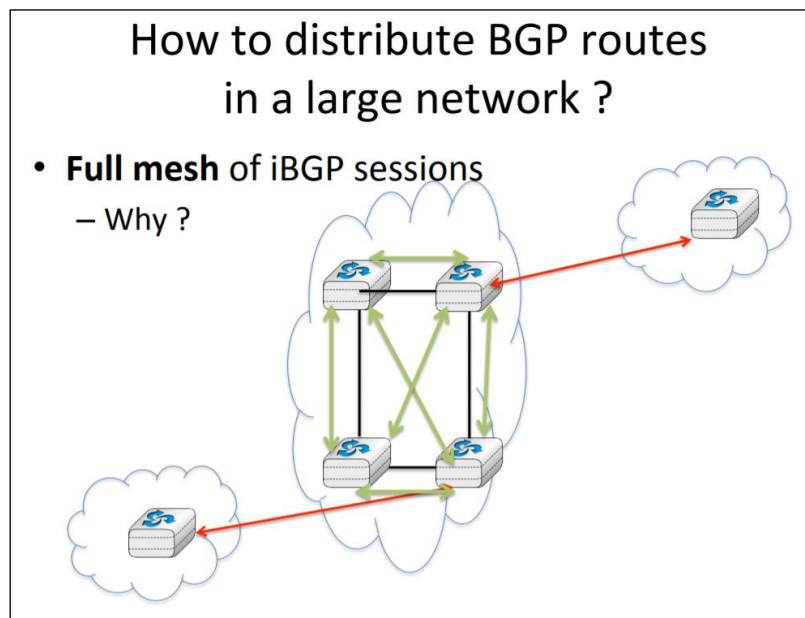
3.1 Principes généraux

- Un routeur (non Route-Reflector) n'advertise pas de route apprise par iBGP, seulement sa meilleure route vers un préfixe apprise directement par eBGP.
- Seule la meilleure route (basée sur les 7 critères décrits plus bas) vers un préfixe est advertised.
- Si un routeur advertise une route puis finalement en apprend une meilleure, il envoie un withdraw pour prévenir les autres routeurs de ne plus utiliser l'ancienne route (uniquement lui va la garder en mémoire).
- Un routeur recevant un message de withdraw pour une des routes de sa table la retire complètement (ne la garde pas en mémoire).
- Pas de filtre d'exportation iBGP (contrairement aux filtres eBGP).

- Lorsqu'on parle de liens iBGP il s'agit, de façon générale, de liens virtuels et non de véritables liens physiques directs. Cela dit, si les liens iBGP suivent les liens physiques c'est d'autant mieux (au niveau des performances).
- Les messages d'iBGP sont envoyés sur les adresses loopback (contrairement l'adresse d'une interface spécifique qui serait inaccessible si l'interface meurt, l'adresse loopback sera accessible tant qu'au moins une des interfaces est fonctionnelle).
- Il peut arriver qu'un paquet ne suive pas exactement le chemin interne qui était attendu, il s'agit alors de « redirection ». Exemple ; Un routeur R1 doit envoyer un paquet vers le préfixe P appartenant à un autre AS. Dans sa table, il dispose d'une route vers P lui disant qu'il doit envoyer vers R6. Or pour envoyer à R6, il doit passer, par exemple, par R5. Il se peut que dans R5, la table de R5 considère que pour atteindre le préfixe P il faut envoyer à R7 et pas à R6. Le paquet suivra alors un parcours différent de celui qui était initialement attendu (du point de vue de R1). Si le réseau est correctement configuré, cela n'est pas un problème.
- Généralement, si pas de configuration particulière (RR) n'est précisée, on considérera que les routeurs sont en full-mesh iBGP.

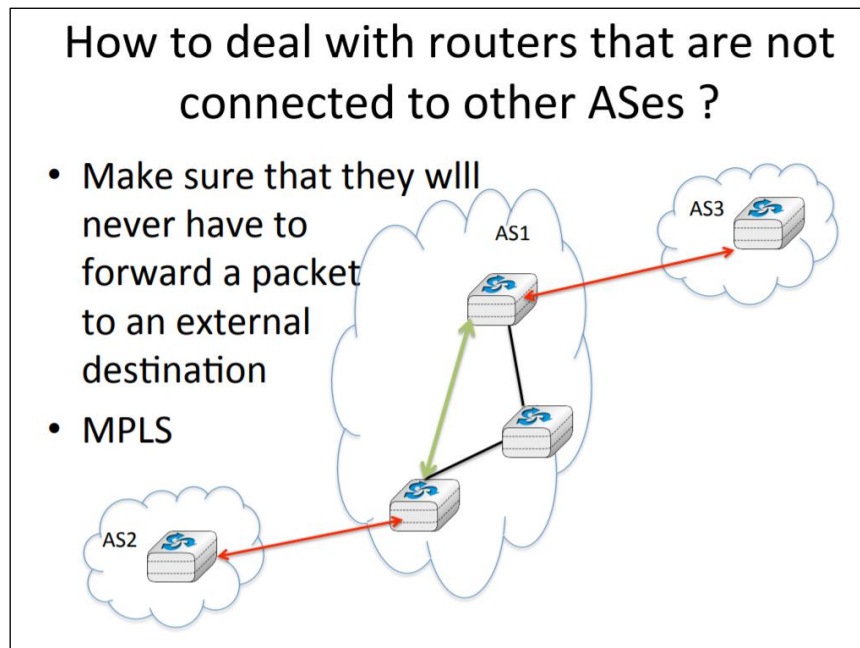
3.2 Cas full-mesh sans utilisation de Route-Reflector

Tous les routeurs sont connectés entre eux. Le routeur apprenant par un lien eBGP une nouvelle route vers un préfixe P advertise cette nouvelle route à tous ses voisins (donc tous les autres routeurs car full-mesh) si cette route est sa meilleure route vers le préfixe P. Tous les routeurs internes ont donc directement cette nouvelle route dans leur table (s'ils n'en ont pas déjà une meilleure). Cette solution est difficilement viable sur un large network car demande un full-mesh et donc énormément de messages et de liens (tous les routeurs sont interconnectés).



3.3 Cas MPLS

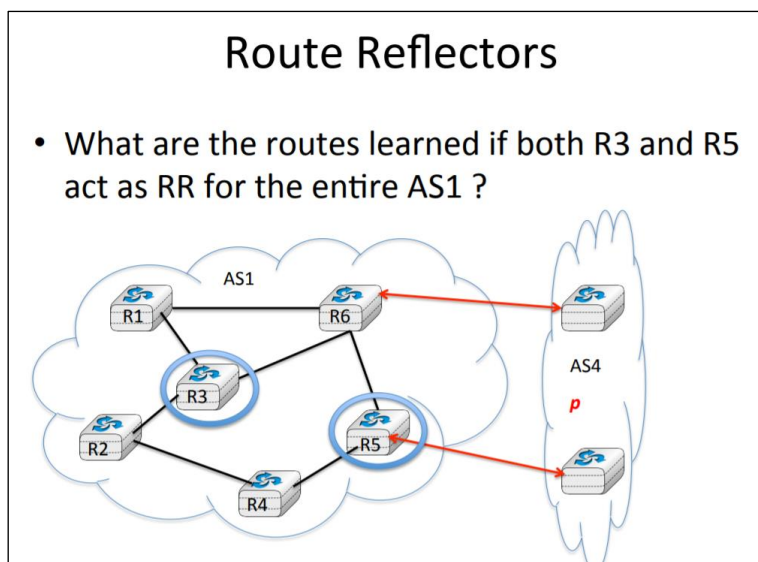
MPLS, qui sera détaillé plus loin, permet de créer des tunnels entre deux routeurs en passant sur un protocole différent du protocole IP (habituel). Une solution peut-être de relier des routeurs de bordure (routeurs avec une session eBGP active) directement entre eux via un tunnel MPLS afin que les paquets devant être forward vers un autre AS n'aient pas à passer par les routeurs internes (les paquets sont dirigés selon leur label sans nécessité pour les routeurs internes d'avoir la route externe dans leurs tables de routing). Ce n'est pas souvent utilisé en pratique car restreint fortement les routeurs non-bordure.



3.4 Cas des Route-Reflectors :

Les routes-reflectors sont des routeurs particuliers qui vont advertir à leurs clients leurs meilleures routes même si elles ont été apprises par iBGP. Pour être plus précis, les Routes-Reflectors ont des liens clients et des liens non-clients. Généralement, les routeurs normaux vont être les clients des RR (les RR sont connectés à tous les routeurs normaux par des liens iBGP) et les RR vont être en liens non-client entre eux (généralement full-mesh iBGP entre les RR). En règle générale, un routeur eBGP recevant une nouvelle meilleure route par la session eBGP va l'annoncer via iBGP aux RR qui vont ensuite l'advertir à tous les autres routeurs (s'il s'agit de leur meilleure route). Pour être plus précis, un RR recevant un BGP update de la part d'un lien non-client (autre RR), l'advertit à tous ses clients. Un RR recevant un bgp update de la part d'un client l'advertit à tous ses autres liens (y compris les liens non-clients). Il est important de noter que lorsqu'un RR advertit ses meilleures routes, il s'agit de ses meilleures routes de son point de vue, ces routes ne sont pas spécialement les meilleures pour ses clients. Cela signifie en pratique que la position des RR a une grande importance dans la configuration d'un network car s'ils sont mal positionnés, les RR peuvent créer des comportements non-voulus (potentiellement des loops). Les RR sont généralement placés par paires dans un network. Les RR peuvent également être organisés en hiérarchie avec plusieurs

niveaux de RR tel que dans le projet, on a alors des RR qui peuvent être clients d'autres RR de niveaux supérieurs. A noter que l'introduction de deux attributs Originator_ID et Cluster_List ont permis de prévenir tout risque de loop lié au positionnement des RR. Originator_Id est set à l'ID du routeur qui a injecté la route dans l'iBGP et le RR ajoute son propre ID dans Cluster_List lorsqu'il advertise la route en iBGP. Il suffit alors au RR de refuser une route lorsque son propre ID est déjà présent dans le Cluster_List.



4. Critères de sélection des routes pour eBGP et iBGP

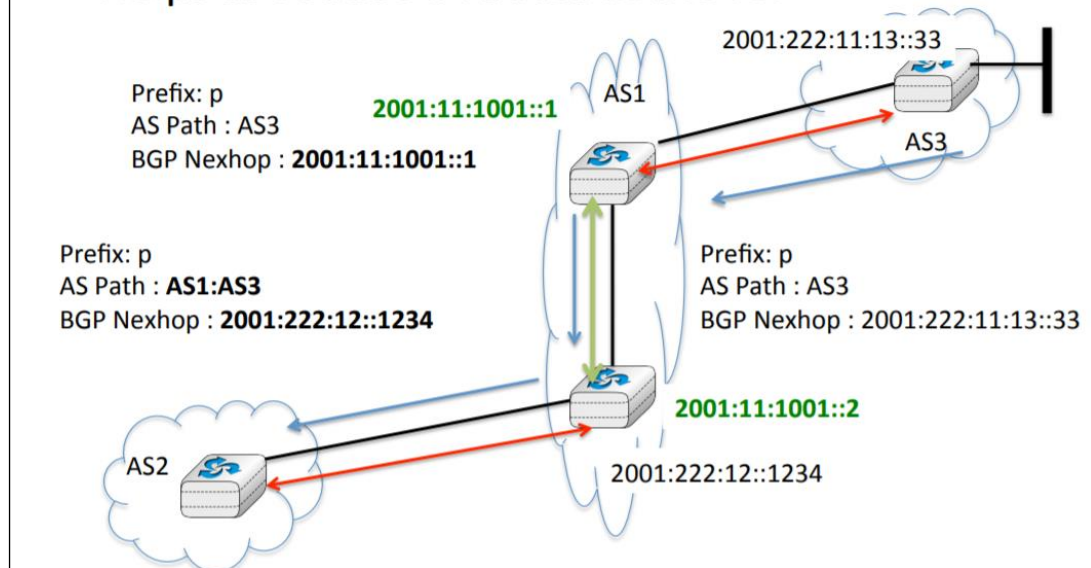
Lorsque je parle de « route préférée » ou de « meilleure route » il s'agit en réalité de la route sélectionnée parmi d'autres routes menant au même préfixe sur base d'une série de 7 critères triés par ordre d'importance. Cela s'applique très souvent en iBGP (par exemple quand RR doit choisir sa meilleure route vers un préfixe P lorsqu'il reçoit deux routes différentes par deux de ses clients menant toutes deux au préfixe P). Mais il s'agit exactement des mêmes critères utilisés également en eBGP (par exemple lorsqu'un routeur eBGP possédant plusieurs sessions eBGP avec différents AS reçoit deux routes différentes menant au même préfixe, par exemple, provenant de deux sessions eBGP différentes).

4.1 Ignorer les routes dont le BGP nexthop est injoignable (dead)

L'attribut Next-Hop est présent dans les messages eBGP ainsi que dans les messages iBGP. Il représente le routeur (adresse) suivant à qui forward le paquet pour atteindre le préfixe concerné. Le nexthop est modifié uniquement quand le paquet bgp passe par un lien eBGP. L'exemple suivant sur image est très visuel concernant l'utilisation en pratique de l'attribut NextHop. Enfin, il peut arriver que le NextHop soit simplement injoignable (dead), il faut alors évidemment abandonner la route en question (withdraw). Mais en pratique lors d'une failure, il peut se passer un certain temps pendant lequel la route n'a pas encore été retirée, on a alors un NextHop injoignable. Il ne s'agit là que d'un état transitoire lors de failures.

BGP Nexthop self

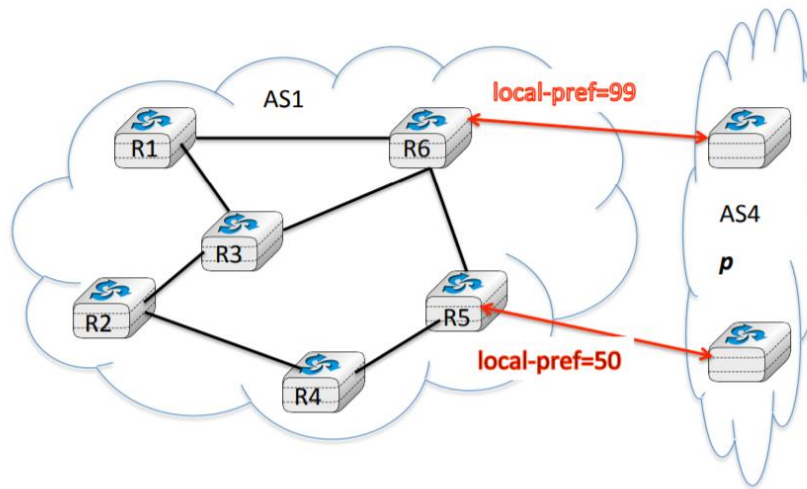
- Helps to reduce # routes in the IGP



4.2 Préférer les routes avec une local-pref élevée

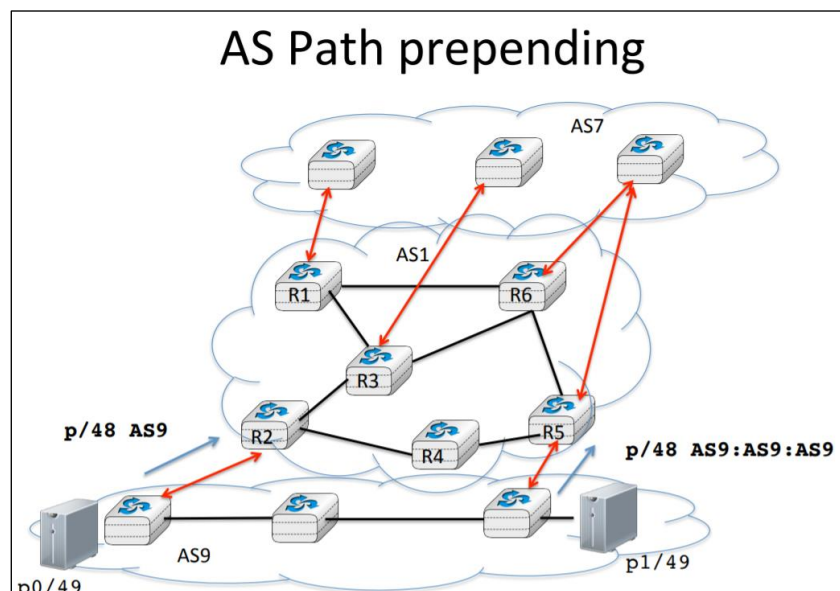
La local-pref est un attribut concernant la configuration interne à un AS qui vise à privilégier un lien eBGP sur un autre. Par exemple, on peut placer une local-pref élevée sur un lien eBGP avec un client pour favoriser ce lien plutôt qu'un lien provider (pour des raisons économiques). En pratique, la valeur de local-pref est généralement définie manuellement ou via les import-filters du routeur eBGP qui entretient la session eBGP en question. La local pref est ensuite distribuée aux routeurs internes dans les paquets iBGP. Il est important de noter qu'un paquet eBGP en lui-même ne possède pas l'attribut local-pref. Autrement dit, si j'incarne un AS, un autre AS avec qui j'entretiens une session eBGP ne peut pas directement définir lui-même la local-pref que j'attribuerai au lien eBGP avec lui (à moins d'utiliser diverses astuces de type eBGP communities, etc...). Sur l'image, par exemple, on constate les local-pref que l'AS1 attribue, de son côté, aux deux liens eBGP avec l'AS4 (l'AS1 préfère utiliser le lien via R6 plutôt que le lien via R5). Nous savons donc cela concernant la politique de AS1 mais il est intéressant de noter que cela ne nous dit pas grand-chose de la politique utilisée du côté de AS4 qui peut potentiellement avoir des local-pref totalement différentes pour ces deux liens).

BGP routes towards prefix p on all routers inside AS1



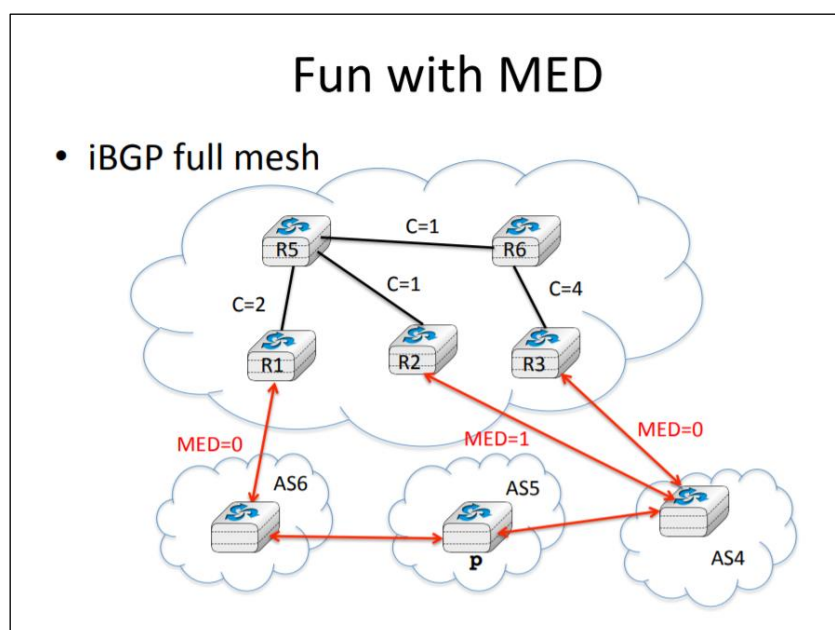
4.3 Préférer les routes avec un AS path plus court

Le AS path représente la suite d'AS qui vont devoir être traversés pour atteindre le préfixe en question. De façon générale, un AS path plus long signifierait devoir traverser plus d'AS différentes et aurait donc potentiellement des performances plus basses qu'un chemin avec un AS path plus court. Il est à noter que ce n'est pas toujours le cas, d'autant plus que l'AS path peut potentiellement être manipulé. Par exemple, un routeur qui adverte un préfixe via une session eBGP peut décider de placer un AS path artificiellement allongé (exemple : AS2,AS2,AS3 au lieu de AS2,AS3) pour défavoriser cette route comparée à une autre. Enfin, il est à noter que l'attribut AS path ne peut être modifié que lors d'un passage par une session eBGP (comme pour le NextHop). A noter qu'une fois l'AS path transmis par la session eBGP, il est également transmis par iBGP mais non-modifié (attribut présent dans les paquets eBGP et dans les paquets iBGP)



4.4 Préférer les routes avec un MED plus petit

MED signifie Multi-Exit Discriminator. En pratique, il s'agit d'un attribut transmis par les paquets d'eBGP (donc provenant de l'AS voisin) indiquant, dans le cas où il y aurait plusieurs liens eBG avec cet AS tous menant aux mêmes préfixes, lequel utiliser en priorité. Il s'agit donc d'une information à prendre en compte dans le processus de choix de route uniquement lorsque plusieurs sessions eBGP sont établies vers le même AS voisin. Une fois le MED transmis par la session eBGP, les messages d'iBGP en interne vont également transmettre le MED (attribut présent dans les paquets d'eBGP et dans les paquets d'iBGP). Une pratique courante pour un AS qui advertise un préfixe par eBGP est de placer dans le MED la distance interne par rapport au préfixe (le cout pour joindre le NextHop depuis le routeur eBGP qui a advertise le préfixe, on note alors MED=IGP). Sur l'image suivante, par exemple, le MED n'est intéressant que lorsqu'on s'intéresse aux liens via R2 et R3, le MED sur le lien R1 n'a aucun intérêt pour la prise de décision.



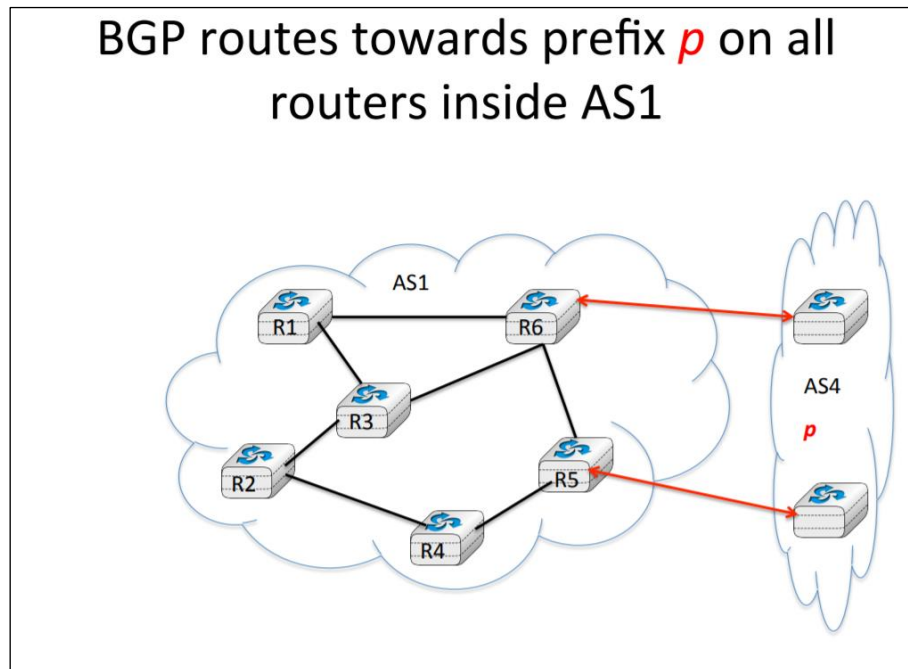
4.5 Préférer les routes apprises par eBGP plutôt que par iBGP

Cela concerne les routeurs eBGP (c'est-à-dire les routeurs possédant au moins une session eBGP active). Ils donnent priorité aux routes apprises par leur lien eBGP plutôt qu'aux routes apprises par leurs voisins internes (iBGP). Cela correspond au principe de hot potato (envoyer le plus rapidement possible vers un autre AS pour être débarrassé du paquet).

4.6 Préférer les routes avec le NextHop le plus proche

Il s'agit là du critère qui sera souvent décisif dans le choix de route apprise par iBGP (car assez souvent les critères supérieurs tels que local-pref ou MED ne sont pas présents ou pas pertinents). Concrètement lorsqu'un routeur reçoit deux routes menant au même préfixe mais passant par deux routeurs eBGP différents (NextHop), il choisit en priorité la route

apprise par le routeur eBGP (NextHop) le plus proche. Sur l'image, par exemple, R1 et R3 préféreront la route apprise via R6 tandis que R4 et R2 (pour R2 il faut également se baser sur le 7^{ème} critère concernant les ID des routeurs) préféreront la route apprise via R5.



4.7 Préférer les routes apprises par le routeur eBGP d'ID le plus bas

Il s'agit là du dernier critère donc à ne prendre en compte que si tous les critères précédents n'ont pas été déterminants. Comme le titre l'explique, il suffit de choisir la route apprise par le routeur eBGP dont l'ID est le plus bas. À noter que l'intérêt ici est de trancher pour finalement ne sélectionner qu'une seule route, il s'agit donc d'un critère qui a été décidé arbitrairement. Il existe malgré tout des routeurs plus récents qui font du load-balancing entre deux routes si elles sont les mêmes attributs BGP (deux routes pour lesquelles les 6 critères supérieurs n'ont pas été déterminants).

5. Multicast

5.1 Principes généraux

L'intérêt est de pouvoir envoyer un paquet à tout un ensemble de receivers. Par exemple pouvoir envoyer un paquet à x receivers sans que le sender n'ait besoin d'envoyer x paquets. On va alors créer des multicast-groups qui représentent des sets de receivers. En pratique, une seule adresse va identifier tout le groupe et lorsqu'un sender voudra envoyer au groupe entier, cette IP multicast address sera placée comme adresse destination. Il n'y a pas de restriction géographique pour l'adhésion à un groupe. Quand un routeur reçoit un paquet avec comme destination une IP multicast address, au lieu de simplement forward le paquet, il va le copier et le dispatch vers plusieurs routeurs construisant un arbre (qui sera détaillé plus

loin). Remarque, TCP n'est pas très adapté pour du multicast mais UDP fonctionne très bien en multicast.

5.2 Modifications au niveau du Sender

Au niveau de la couche IP, il faut utiliser un flag pour informer que la destination n'est pas une simple adresse identifiant un unique host mais qu'elle représente tout un groupe. Au niveau de la couche Ethernet, du broadcast n'est pas adapté (atteindrait tous les routeurs, même ceux pas intéressés). La solution consiste en un mapping entre les adresses de IP Multicast et les adresses Ethernet (les 23 bits de poids faible de l'adresse Multicast sont copiés dans les 23 bits de poids faible de l'adresse ethernet).

5.3 Modifications au niveau du Receiver

Au niveau de la couche IP, il faut indiquer que le routeur est intéressé par les paquets destinés à un certain groupe en particulier. Pour cela, il faut utiliser le protocole IGMP.

5.4 IGMP

IGMP signifie Internet Multicast Group Protocol. Il s'agit d'une interaction entre un host receiver et son routeur direct auquel il est connecté afin de le prévenir qu'il est intéressé par les paquets liés à un certain groupe (autrement dit, l'host en question fait partie de ce groupe). Etant donné qu'il s'agit d'une interaction au niveau local, ce protocole ne cause pas d'impact global à grande échelle. Plus précisément, l'host receiver envoie une join request au routeur qui va retenir qu'un de ses hosts fait partie d'un certain groupe (groupe identifié par une IP multicast address). Le routeur va cependant retenir cela pendant une durée limitée, il est donc nécessaire pour le receiver de renvoyer des messages au routeur régulièrement pour rafraîchir son state. Concernant ce qui est en mémoire dans le routeur, il y a deux possibilités. Soit le routeur a un compteur lui indiquant le nombre d'host intéressé par un groupe en particulier. Si ce compteur atteint 0, il retire ce groupe de sa mémoire. Soit il utilise un flag qui indique simplement qu'au moins un host est intéressé par un groupe en particulier. S'il reçoit un leave message de la part d'un de ses hosts, il ne sait pas alors s'il a encore des hosts membres du groupe ou non et doit envoyer des query à ses hosts pour s'en informer. Cela demande donc plus de communications (messages) mais moins de mémoire pour le routeur.

5.5 ASM

ASM signifie Any-Source IP Multicast. Une adresse ASM identifie un groupe de receiver. N'importe qui peut envoyer un paquet à un groupe de receivers et n'importe qui peut rejoindre un groupe de receivers. Cela représente un gros manque de sécurité. De plus, les adresses créées pour identifier un groupe de receiver sont générées de façon relativement

random en espérant qu'elle ne soit pas déjà utilisée par quelqu'un (risque de collision). Et enfin, tous les senders sont traités de la même façon même si un est connu est très régulier (un système plus performant pourrait être mis en place pour les senders réguliers).

5.6 SSM

SSM signifie Source-Specific IP Multicast. Une adresse SSM identifie un groupe de receivers liés à un sender spécifique. Une adresse SSM avec l'adresse de son sender identifie de façon unique un channel SSM. Seulement le sender spécifiquement autorisé peut envoyer un paquet au groupe de receivers. Les receivers savent qui est le sender, ils peuvent alors simplement demander à joindre le channel lié à tel ou tel sender. Les paquets sont forwardés en se basant sur l'adresse de destination (channel) mais également en se basant sur la source (contrairement au ASM). Pour rejoindre et localiser les hosts qui adhèrent aux channels, on utilise du IGMP (même principe qu'en ASM mais les hosts précisent la channel (adresse) ainsi que la source). Ensuite concernant l'acheminement des paquets depuis la source vers les receivers, un graphe sans loop, source-tree, est progressivement construit pour relier la source à tous les receivers. Chaque routeur multicast devra alors garder en mémoire pour chaque sender une interface in et une liste d'interfaces out (sur lesquels copier et envoyer les paquets). Remarque, si deux sources envoient vers la même SSM adresse, cela va créer deux source-trees différents (deux channels uniques identifiés chacun par leur SSM adresse (commune) et leurs sources (différentes)) contrairement au ASM qui aurait créé un seul tree.

5.7 Methodes de construction de tree

Pour construire les différents arbres (tree), plusieurs méthodes existent, certaines plus adaptés au ASM, d'autres plus adaptées au SSM et parfois utilisable dans les deux cas.

Multicast OSPF (MOSPF) : Une première approche est de construire un arbre en se basant sur le protocole OSPF. Via IGMP nous savons quels routeurs sont membres de groupes multicast, cette information est ensuite transmise à travers les Link-State Packets échangés par OSPF. Chaque routeur crée alors son spanning tree via dijkstra. Cependant, cela signifie énormément d'échanges de paquets (link-state packets) mais également des algorithmes coûteux (dijkstra) tournant sur chaque routeur. En pratique, cette solution n'est pas utilisée ni pour ASM ni pour SSM.

Broadcast Multicast : Une deuxième approche (aussi appelée Reverse Path Forwarding) est d'envoyer les paquets partout (le premier paquet vers un groupe est envoyé à tout le monde). Si un routeur reçoit un paquet qui ne l'intéresse pas, il prévient son upstream (voisin qui lui a envoyé le paquet) de ne plus lui transmettre ces paquets. Pour éviter trop de retransmissions inutiles, quand on reçoit un paquet (on peut donc voir sa source), s'il arrivé par une interface qui n'est pas celle qu'on aurait utilisée pour joindre sa source, on discard le paquet (car il existe un chemin plus court et le paquet va donc arriver aussi par une interface plus intéressante). De plus, si on a déjà reçu un paquet d'un voisin direct, il n'y a plus besoin de lui

envoyer de paquet (car on n'est pas sur son shortest-path entre lui et la source). Ces dernières décisions réduisent grandement le nombre de communications nécessaires mais peuvent amener à des arbres non-optimaux notamment dans le cas de liens asymétriques mais c'est un coût qu'on décide de prendre malgré tout pour faciliter la construction de l'arbre. Cette technique de création d'arbre est particulièrement utilisée en ASM d'autant plus lorsqu'il y a peu de trafic mais un très grand nombre de receivers. Cette technique peut potentiellement être utilisée également pour du SSM mais cela n'est pas optimal car connaissant la source unique il est possible de créer un arbre plus performant.

ASM direct source-tree : Une méthode très simple en pratique pour du SSM est que, vu que chaque receiver connaît la source qui l'intéresse, il envoie un join message à la source en suivant leur table de routing, correspondant donc à leur chemin le plus court (vers la source, pas depuis la source mais c'est un désavantage qui va être accepté malgré tout). Cela crée du coup très facilement une branche de l'arbre reliant le receiver à la source (root du tree). Pour quitter le groupe, il suffit d'envoyer un leave message à la source. Cela demande beaucoup moins de communications que pour les techniques précédentes et est utilisé en pratique pour SSM.

PIM Sparse Mode : Cette méthode se base sur l'idée de point de rendez-vous. Les senders vont envoyer leurs paquets au point de rendez-vous et celui-ci va ensuite transmettre à tous les receivers. Quand le routeur rendez-vous reçoit un paquet du sender, il connaît son adresse, cette méthode peut donc être adaptée pour du SSM mais de base elle est surtout utilisée en ASM. Le sender ne sait pas spécialement que les receivers utilisent un point de rendez-vous (configuration PIM) mais ce n'est pas un problème car quand un routeur recevra un paquet de la source (par exemple un paquet passe chez lui en forward), étant donné que le routeur sait qu'il est en configuration PIM, il forwardera le paquet vers le point de rendez-vous. Le point de rendez-vous (root) saura alors qu'il y a un nouveau sender et forwardera le paquet à tous les receivers concernés. Le routeur root étant conscient qu'il y a un nouveau sender, il envoie un message vers le sender, créant ainsi une branche (source-tree) entre le point de rendez-vous et le sender. On a un shared-tree entre les receivers et le rendez-vous point et un source-tree entre le rendez-vous point et le sender. En réalité ce système de point de rendez-vous n'est utilisé qu'au début pour découvrir des senders. Une fois que les receivers ont reçu des paquets du sender (via le point de rendez-vous), ils envoient généralement un prune message au root (rendez-vous point) pour ne plus recevoir les paquets de ce sender et créent eux-même une branche (source-tree) jusqu'au sender (dont ils connaissent désormais l'IP). Tous les routeurs en configuration PIM doivent connaître le point de rendez-vous, pour ce faire, cette information est distribuée au sein des messages OSPF ou via broadcast. Il est possible d'avoir plusieurs points de rendez-vous au sein d'un sous network mais cela sort du cadre de ce cours.

6. MPLS

6.1 Principes généraux :

MPLS signifie MultiProtocole Label Switching. Etant donné que les tables de routing des routeurs ont tendance à devenir énorme (près d'un million de lignes pour ipv4), il est intéressant de pouvoir forward des paquets plus rapidement par-dessus la couche IP. C'est ce que propose MPLS. Plutôt que de devoir chercher dans la grande table de routing en prenant le longest match, on va avoir une petite table de Labels (Label Forwarding Table, aussi notée LFIB) qui, pour chaque Label connu input, va définir sur quelle interface forward le paquet et quelle opération faire sur le label (swap par un autre label, pop pour retirer le label, push pour rajouter un label). MPLS est un protocole qui va run au-dessus de IP, en plaçant un petit header spécial au-dessus du header IP classique. Ce header (potentiellement plusieurs header superposés car incarnant une stack de labels) ajouté, de 32 bit, dispose de 20 bits pour le label. Un routeur border peut par exemple ajouter un label, ce qui va transformer le paquet IP en paquet MPLS. Les autres LSR (Label Switching Router) vont forward le paquet généralement en opérant des swap sur le label et typiquement un autre routeur border va terminer par un POP afin de retirer le Label (dernier Label retiré du coup le champ TTL du label header est copié dans le champ TTL du header IP et le(s) header(s) label est (sont) entièrement retiré(s)) et ainsi refaire passer le paquet sur le protocole IP. Il existe deux façons d'organiser et concevoir les labels dans un LSR (Label Switching Router), soit par interface ce qui signifie que pour chaque interface il faudra définir la liste de labels et comment les interpréter. Soit une liste de labels par routeur, peu importe l'interface par laquelle le label est arrivé. C'est cette deuxième solution qui sera plus souvent utilisée notamment car elle permet plus de flexibilité notamment lorsque nous aborderons les failure-recovery et loop-free alternates. LSP signifie Label Switching Path et désigne le chemin suivi par un paquet MPLS commençant par un ingress LSR qui ajoute le Label (sur base d'une table de mapping indiquant quel label push selon le préfixe à joindre) et terminant par un egress LSR qui fait le POP final. De base, un LSP est généralement unidirectionnel.

6.2 Distribution des Labels

Il est nécessaire pour utiliser du MPLS que les LSR aient connaissance des Labels et de leurs interprétations (tables LFIB). Pour distribuer les Labels, une solution simple est d'utiliser un protocole de routing déjà existant sur le network pour peu qu'il soit extensible tel qu'OSPF ou iBGP en ajoutant les informations concernant les Labels dans les paquets. Cependant, deux méthodes en particulier vont être détaillées.

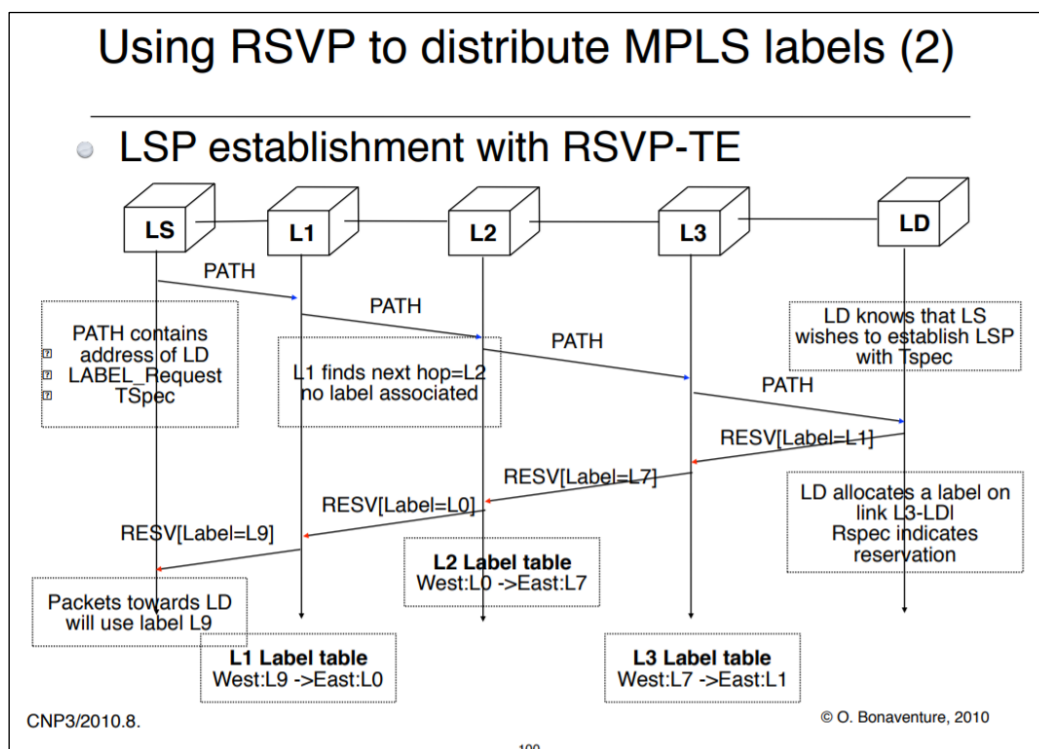
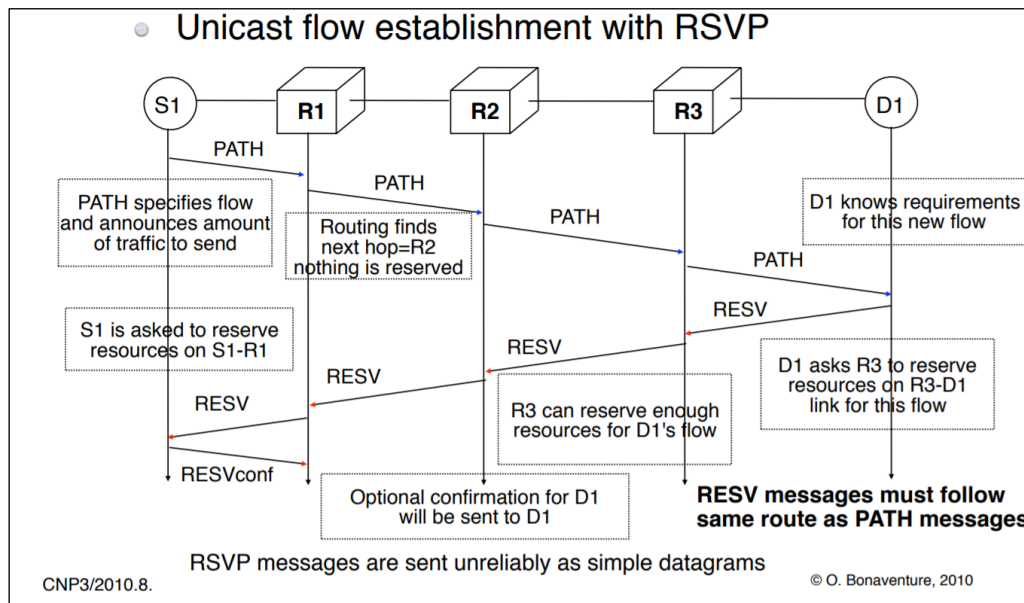
LDP :

LDP signifie Label Distribution Protocol. Ce protocole a été créé pour distribuer les Labels sur base de hop-by-hop (voisin à voisin) quand les Labels ne peuvent pas être distribués via un autre protocole de routing déjà existant sur le network en question. Des messages UDP sont envoyés aux voisins directs pour déterminer s'il s'agit de routeurs normaux ou de LSR. Les LSR établissent ensuite des sessions LDP entre eux échangeant des informations différents

messages liés aux Labels : tels que Label mapping pour annoncer un label ou Label withdraw pour retirer un Label. En partant d'un LSR qui souhaite créer un LSP (tunnel) vers un autre LSR, LDP va distribuer les labels en suivant le chemin le plus court depuis la source pour atteindre la destination. Dans le cas de link-failure, ospf recalcule le chemin le plus court entre les routeurs (scénario normal en cas de link-failure) et la LFIB est alors mise à jour.

RSVP-TE :

RSVP signifie Ressource Reservation Protocole, TE signifie Traffic Engineering, il s'agit donc d'une combinaison des deux qui permet en réalité de créer des tunnels LSP (Label Switching Path) tout en leur réservant de la bande passante. Ce protocole comporte principalement deux messages importants : PATH utilisé par le sender pour demander un Label et des ressources (bande passante) tout en spécifiant le chemin qu'on souhaite créer pour le LSP (le chemin complet est écrit, il s'agit d'explicit routing). RESV envoyé par le receiver pour confirmer qu'on a réservé des ressources pour le chemin décrit dans le Path reçu précédemment. Ces messages RSVP passent par des paquets IP et doivent suivre le même chemin que celui décrit par le PATH (après avoir reçu les PATH, le receiver final envoie un RESV message au routeur précédent lui annonçant qu'il a bien dédié une certaine bande passante pour ce Path, ce routeur envoie ensuite un message RESV au routeur précédent et ainsi de suite jusqu'à remonter au sender). RSVP utilise un soft-state, c'est-à-dire que les messages de PATH et RESV sont retransmis de façon régulière sans quoi les réservations périment et sont oubliées. Si un routeur intermédiaire crash, le state est automatiquement reset au bout d'un court laps de temps. Si une route change et qu'une nouvelle réservation est établie (pour la nouvelle route modifiée), l'ancienne route ne reçoit plus de messages RSVP et est rapidement expirée. Malgré cela, pour ne pas perdre de temps, des messages de end of flow (tear) existent afin de reset le state plus rapidement qu'en attendant le time out. En pratique, RSVP est utilisé pour distribuer les Labels de la façon suivante, le sender fait envoi une requête PATH vers le receiver, le processus RSVP est utilisé avec les routeurs intermédiaires qui communiquent comme décrit plus haut. Les messages RESV de retour, partant du receiver allant progressivement vers le sender comportent entre autres le Label qui sera utilisé. Enfin, l'aspect traffic engineering (TE) vient des extensions qu'on peut utiliser avec RSVP pour facilement permettre une large variété d'options telles que réserver un PATH dont tous les liens sont d'au moins x Mbps, un PATH dont aucun lien dépasse 10 ms de délais, un PATH disjoint d'un autre PATH spécifique, etc... Pour cela, il suffit que le routeur sender qui envoie le message RSVP PATH initial (comportant le chemin complet) calcule le chemin en faisant un dijkstra sur base d'un graphe sur lequel tous les liens non-acceptés (ne correspondant pas aux critères) sont retirés. Cela permet alors la création de tunnels MPLS avec des caractéristiques bien spécifiques. Il est important de noter qu'avec RSVP-TE, le sender peut spécifier exactement le chemin qu'il veut, forçant l'utilisation de liens coûteux s'il le souhaite (un peu comme du adjacency routing).

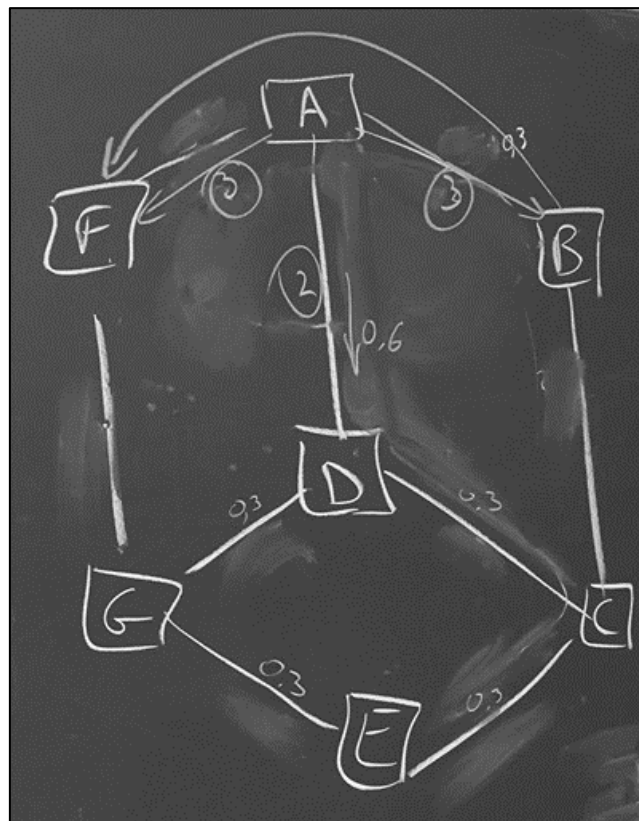
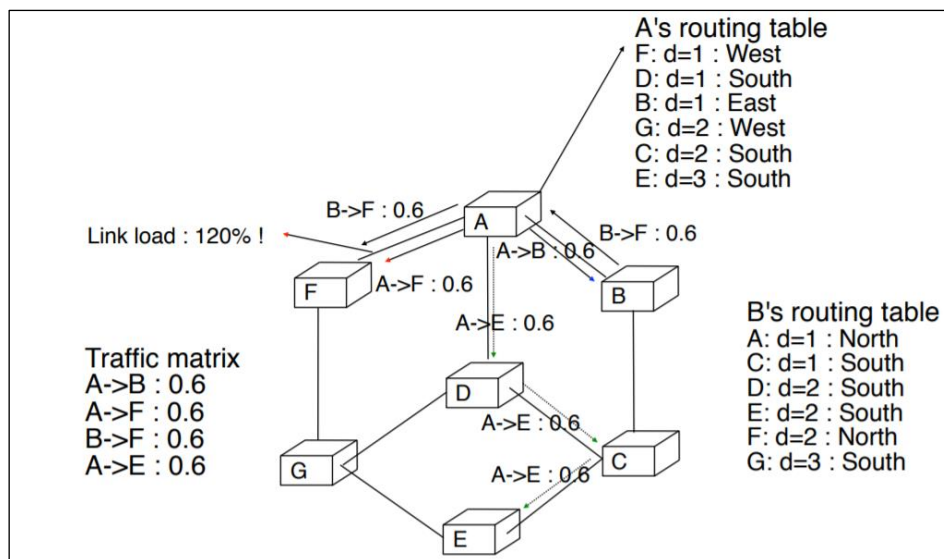


7. Load Balancing

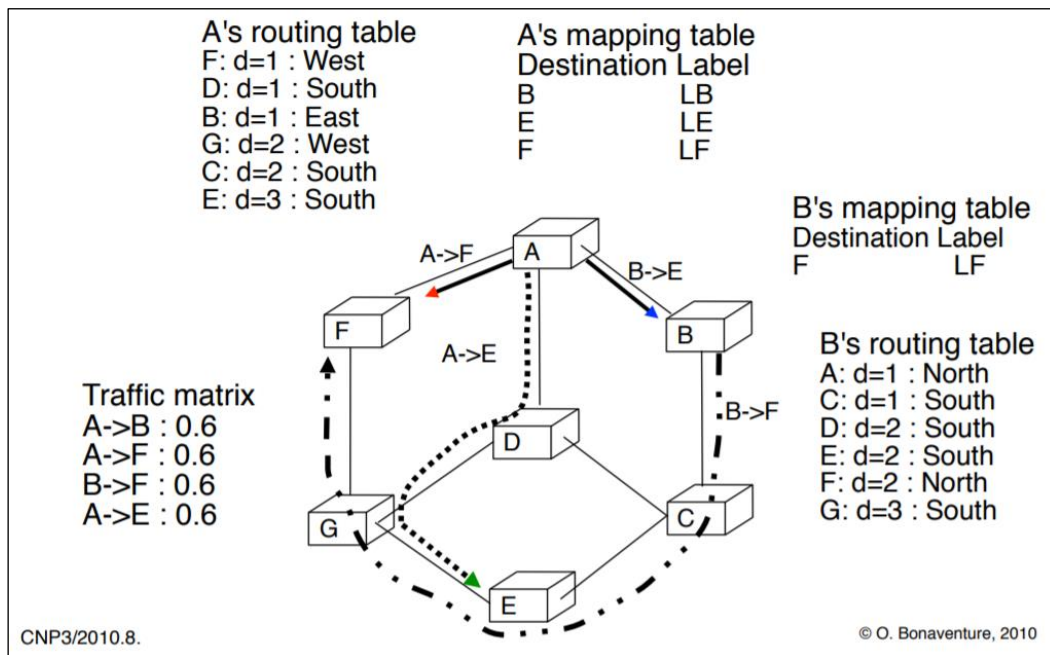
Comme déjà vu précédemment lorsque iBGP et eBGP étaient détaillés, plusieurs solutions existent pour faire du load balancing sur des liens entre AS (AS-path artificiellement allongés, préfixes plus ou moins précis, MED, local-pref, bgp communities, etc...). Mais il est important également d'avoir des stratégies de load balancing en interne dans notre propre network. Pour cela quelques méthodes existent et vont être rapidement explicitées.

Une première solution, comme brièvement mentionnée dans le point 4.7, est d'avoir plusieurs chemins internes de même coût reliant deux routeurs. En effet, les routeurs récents plutôt que de choisir la route apprise par le routeur d'ID le plus bas, vont plutôt faire du load-balancing entre les deux routes. Jouer sur les coûts des liens interne peut donc être une

solution pour permettre du load balancing. Comme on peut le voir sur ces deux images, modifier les coûts sur trois liens a permis de répartir le trafic facilement.



Une deuxième approche est de définir des tunnels LSP via MPLS prenant des chemins différents. En distribuant les Labels via RSVP-TE (pas via LDP car LDP suit systématiquement les chemins les plus courts), cela permet de forcer des chemins différents de ceux qui auraient été définis si on avait choisi les chemins les plus courts de façon systématique (routing de base).



Une troisième technique est de faire du load-balancing par TCP connexion. L'idée est, dans le cas de deux chemins de même coût, d'envoyer une session TCP par un chemin et une autre session TCP par un autre chemin. Mais tous les paquets d'une même session TCP passent par le même chemin pour éviter de devoir re-order les paquets non-stop etc. Pour faire cela, au niveau des routeurs, un hash est calculé sur base des infos du paquets TCP (IP source, port source, IP destination, port destination et TTL). Sur base de ce hash (par exemple s'il est pair ou impair), le routeur va définir si le paquet doit être envoyé sur un chemin ou un autre. A noter que le TTL est inclus dans le hash afin d'éviter le problème de polarisation. Pour illustrer ce qu'est le problème de polarisation, voici un petit exemple si on n'utilisait pas le TTL dans le hash : Imaginons qu'un paquet lié à une connexion TCP R0-R10 passe par un routeur R1, qui a deux chemins de même coût pour atteindre R10, lui calcule un hash pair et décide d'envoyer sur un certain chemin sur base de la parité. Le routeur suivant R2, s'il a deux chemins de même coût vers R10, calculera le même hash et prendra donc une décision sur base de ce hash, le routeur suivant R3, s'il a deux chemins de même coût vers R10, calculera également le même hash, etc... Du coup toutes les connexions TCP vers R10 arrivant au routeur R1 et dont le hash est pair suivront exactement le même chemin, réduisant fortement le load-balancing. En plaçant le TTL dans le hash, chaque routeur sur le chemin qui calcule le hash peut avoir un hash différent et donc prendre des décisions différentes, ce qui signifie que toutes les connexions TCP vers R10 arrivant à R1 ne suivront plus exactement toutes le même chemin, permettant un bien meilleur load-balancing.

8. Failure Recovery

8.1 Failure detection

La première étape pour du failure recovery rapide est de détecter le link failure très rapidement. Pour cela il existe plusieurs possibilités telles que les suivantes.

Physical layer : Dans certains cas, on peut rapidement détecter qu'un lien est mort physiquement par exemple pour de la fibre optique (on détecte instantanément si le signal passe ou non).

Routing Protocol : La plupart des protocoles de routing tels qu'OSPF envoient des messages hello à intervalles réguliers. La non-réception de ces messages après un certain laps de temps signifie simplement que le lien est mort. Mais cette approche est assez lente et peut parfois prendre jusqu'à 10 secondes avant de détecter la panne !

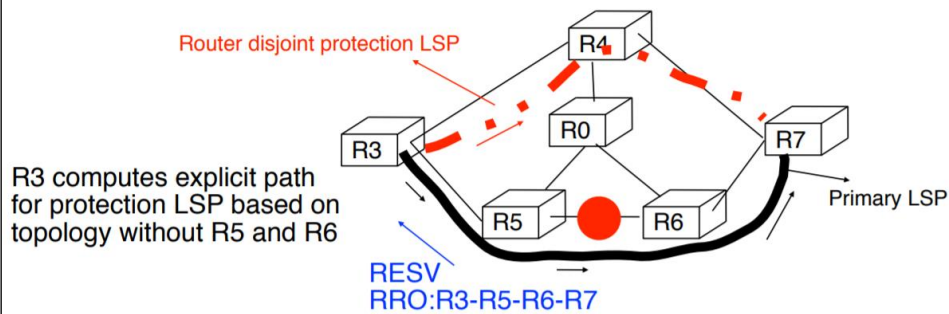
BFD : BFD signifie Bidirectional Forwarding Detection. Il s'agit de paquets UDP retransmis environ tous les 1ms pour vérifier que la connexion est up.

8.2 Failure Fast Recovery

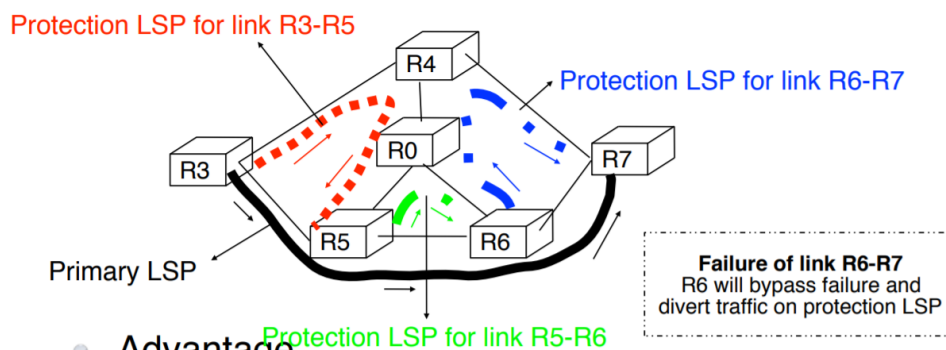
Lorsqu'une link-failure est détectée, il est important de trouver rapidement une solution pour que les paquets soient correctement forward malgré la panne locale. Pour cela, plusieurs solutions existent, certaines très générales, d'autres plus spécifiques à certaines situations.

MPLS secondary LSP : Pour chaque tunnel MPLS (= chaque LSP = chaque Label Switching Path), un deuxième LSP est créé (même méthode que pour un LSP classique), utilisé uniquement en cas de backup. Quand un routeur ou lien du LSP est mort et détecté, le routeur ayant détecté l'erreur envoie un message au routeur sender du LSP (ingress LSR, celui à l'entrée du tunnel LSP) pour le prévenir d'utiliser le LSP secondaire au lieu du primaire. Concernant la création du LSP secondaire, le principe est simple, le ingress LSR calcule le chemin en excluant le chemin utilisé pour le LSP primaire. Ainsi, il obtient un deuxième chemin disjoint du premier. Cette solution possède malgré tout plusieurs défauts. Premièrement, pour chaque LSP existant, il en faut un deuxième en backup, ce qui signifie plus d'utilisation de mémoire pour les routeurs et de paquets RSVP. Deuxièmement, le LSP secondaire (backup) doit impérativement utiliser des ressources différentes de celles utilisées pour le LSP primaire (réservation de beaucoup de bande passante, etc). Et enfin, il peut s'écouler un laps de temps non-négligeable entre le moment où la panne est détectée et le moment où le ingress LSR en est informé. Ce dernier problème peut être corrigé par une alternative qui a, elle aussi, ses faiblesses. En effet, il est possible pour chaque LSR du LSP de définir un petit LSP secondaire pour rejoindre ses voisins. Cela signifie que le détournement sur lien backup se fait au niveau local mais demande un très grand nombre de LSP secondaires. En pratique, pour cette solution, la plus intéressante, chaque routeur LSR a 2 routes distinctes vers ses voisins directs, donc deux lignes dans la LFIB, l'une normale et l'une utilisée en cas de backup. L'idée est la suivante, le LSR détectant la panne swap normalement le label reçu (comme s'il n'y avait pas de panne) et push par-dessus le label du chemin backup vers son voisin (nommons le R2) puis envoie le paquet en suivant le chemin backup. Le paquet sera alors forward sur ce chemin là et le dernier LSR à recevoir le paquet avant d'arriver au LSR R2 va pop le Label de backup pour que R2 reçoive le paquet comme si de rien n'était. Cette solution nécessite d'utiliser une répartition per LSR des Labels et pas une répartition per interface.

- **Types of end-to-end protection LSPs**
 - router-disjoint protection LSP
 - does not utilize the same transit routers as the primary
 - link-disjoint protection LSP
 - does not utilize the same links as the primary LSP



- **Principle**
 - one protection LSP is used to protect each link



- **Advantage**
 - traffic is immediately switched to secondary LSP
- **Drawback**
 - a large number of protection LSPs may be required

Loop-free alternates : Il s'agit là d'une solution très utilisée en pratique et fonctionnant sur tout network qui tourne le protocole IP. Lorsqu'un lien est mort, le routeur détectant la panne (donc un des liens sur une de ses interfaces est mort) redirige directement tous les paquets qui devaient passer par ce lien vers le loop-free alternate (les loop-free alternates sont precomputed), le paquet n'est pas modifié il est tout simplement forward sur l'interface qui joint le loop-free alternate. Un loop-free alternate pour un lien L1 est un voisin direct chez qui on peut rediriger le trafic qui devait passer par ce lien L1 (lien L1 étant mort) sans que ce dernier ne nous renvoie le trafic en retour (donc sans créer de boucle). Parfois pour certains liens, il n'existe pas de loop-free alternate. Il faut alors trouver une solution alternative telle que ; créer un tunnel MPLS pour protéger le lien en question, assigner des adresses spéciales « not via address » dans les tables de routing des routeurs pour éviter tel ou tel routeur dans le chemin ou enfin, utiliser des routeurs non-voisins directs comme loop-free alternates via l'utilisation de segment routing (détaillé en section 10). Pour détailler un peu plus ces solutions, les « not via address » sont des adresses pouvant être incluses dans les tables de routing et qui représentent le fait de joindre une certaine adresse tout en évitant une autre adresse. Par exemple BnvA (B not via A) représente le fait de joindre B en évitant de passer par A. Ce type d'adresse peut être utilisé en destination pour protéger les liens qui n'ont pas

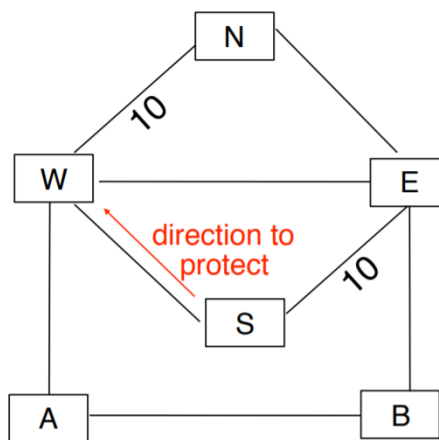
de loop-free alternates mais cela demande plus de lignes dans les tables de routing. Enfin, la dernière solution concernant les loop-free alternates non voisins directs consiste à rediriger le trafic vers un autre routeur alors que celui-ci n'est pas voisin direct. Pour éviter que le voisin direct qui reçoit le paquet ne le redirige vers nous (car aucun voisin direct n'est loop-free alternate), il faut utiliser du segment-routing (modifier le paquet) afin de forcer le chemin suivi par le paquet jusqu'au loop-free alternate distant. Le principe de segment routing sera détaillé dans la dernière section de ce document. Remarque, une structure de network en triangle permet de trouver plus facilement des loop-free alternates et de plus facilement protéger les liens qu'une structure en carrés.

- **Example**

- all links have weight=1 except NW and SE

W's routing table

N : East via E
 S : SouthEast
 E : East
 A : South
 B : South via A
 East via E



E's routing table

N : NothWest
 S : SouthWest
 W: West
 B : South
 A : South via B
 West via W

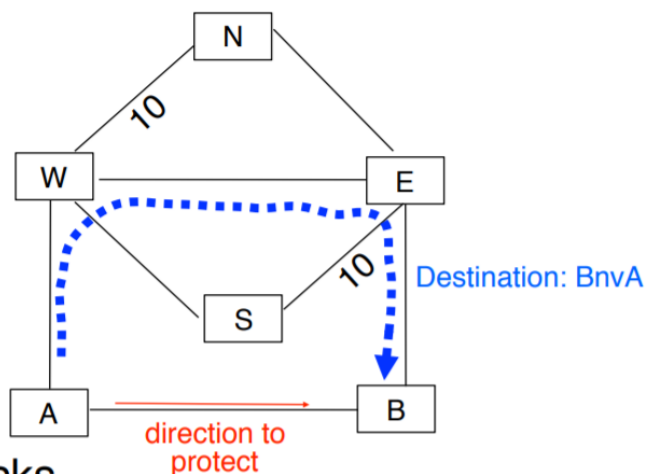
S's routing table

All : via W

- ❓ If S->W fails, E is a loop-free neighbor
- ❓ all S->W's packets sent to E will not loop

Usage of not-via-addresses

- Used as destination addresses for protection tunnels



- **Drawbacks**

- Increases number of addresses
- Increases number of SPF computations

Failures

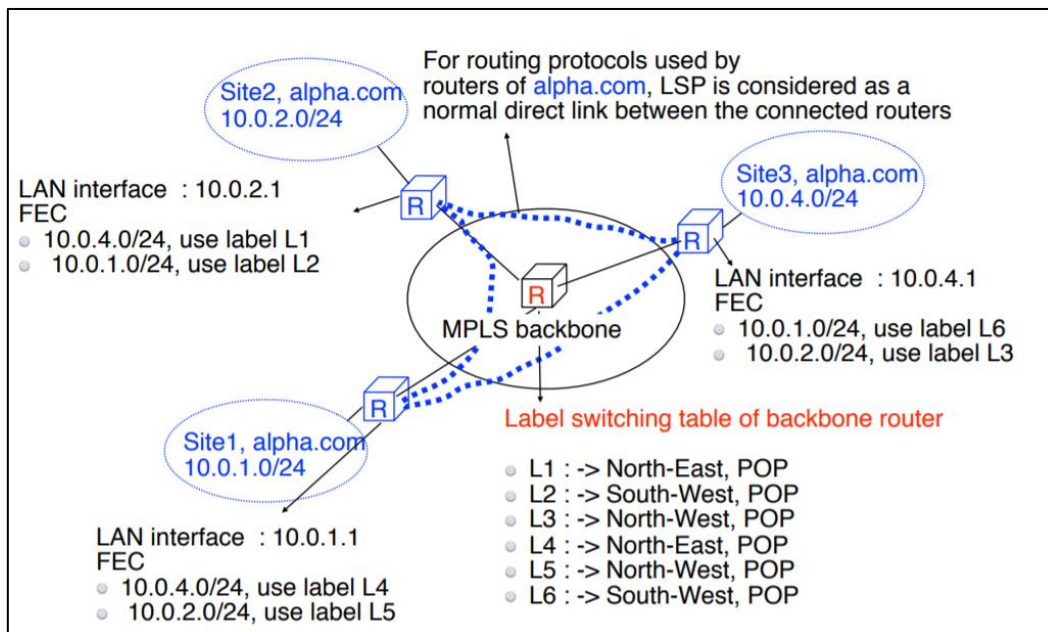
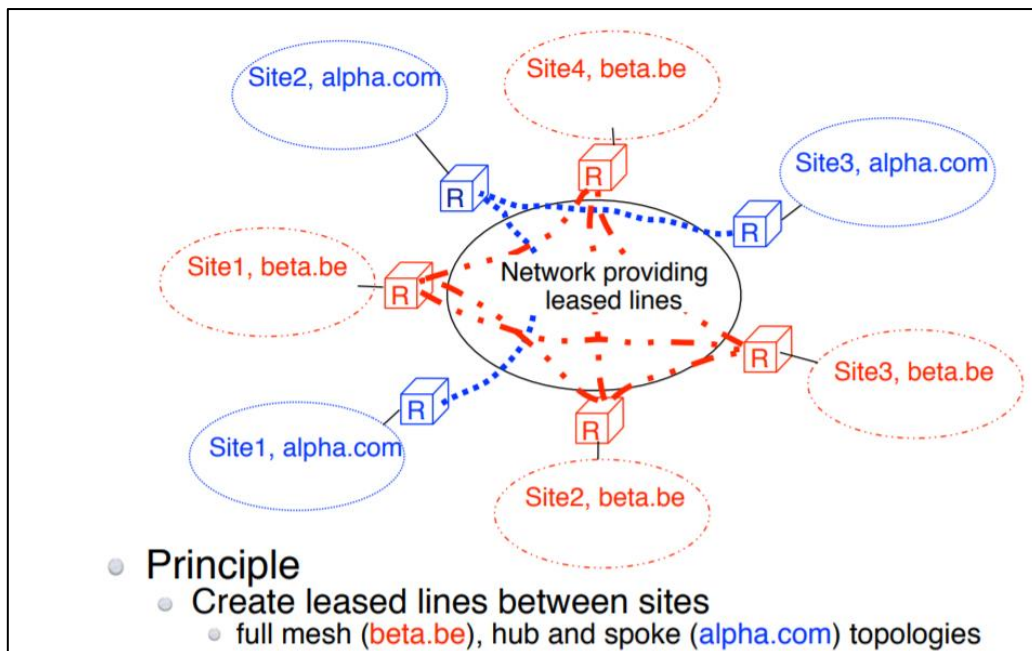
© O. Bonaventure

Remarque concernant la stabilisation après failure recovery : Imagions par exemple qu'un tunnel MPLS ait été créé pour couvrir une link-failure, permettant au trafic d'être correctement forward. Il faut bien à un moment retirer ce tunnel car sinon il y aurait, avec le temps, des tunnels MPLS absolument partout. Il faut donc retirer ce tunnel MPLS une fois que OSPF a recalculé la topologie en prenant en compte le lien supprimé (dans le cas où il ne serait pas très rapidement redevenu vivant). Mais à quel moment exactement retirer le tunnel MPLS ? Car OSPF tourne sur tous les routeurs et ils calculent chacun leur dijkstra, il se peut que ce soit déjà terminé sur un routeur, par exemple sur le routeur ingress du tunnel MPLS mais que d'autres routeurs n'aient pas encore convergé OSPF et aient donc encore besoin du tunnel MPLS pour l'instant. Il n'existe pas de technique standardisée idéale pour régler ce problème mais en pratique on décide de retirer le tunnel lorsque plus aucun paquet ne le traverse pendant un certain laps de temps. En effet, quand OSPF a convergé, les routeurs savent que l'ancien lien n'est plus là et donc ils ne l'utilisent plus, le tunnel de secours n'est donc plus utilisé. Certaines paquets seront peut-être perdus si certains routeurs n'avaient pas encore terminés leur dijkstra mais cela signifiera simplement quelques retransmit, ce qui est acceptable.

9. VPN

9.1 Principes généraux :

VPN signifie Virtual Private Network. Le principe est de permettre à plusieurs sites distants d'agir comme s'ils étaient un unique network virtuel, un VPN. Il peut potentiellement y avoir plusieurs VPN sur un même network réel, par conséquent chaque VPN doit être isolé des autres et véritablement agir comme s'il était seul. De plus, chaque VPN utilise des adresses virtuelles lui étant propres. Il est donc possible qu'un network supporte deux VPN utilisant tous-deux les mêmes adresses virtuelles (deux adresses virtuelles identiques mais appartenant à deux VPN différents font donc référence à des routeurs différents). Une première solution existe consistant à configurer manuellement 2 par 2 les routeurs des sites distants pour créer des full mesh entre routeurs d'un même VPN. Mais cela demande beaucoup de configuration, beaucoup de liens pour le full-mesh et beaucoup de modifications à apporter pour retirer ou ajouter un VPN. Il ne s'agit donc pas d'une solution viable. Une deuxième approche un peu plus intelligente est de créer des tunnels MPLS entre tous les routeurs d'un même VPN. Mais cela requiert que tous les routeurs du VPN supportent MPLS et que tous les routeurs du backbone (partie centrale qui relie les différents sites du VPN entre eux) supportent également MPLS et que tous aient reçus les Labels. De plus, chaque nouveau VPN nécessitera une reconfiguration de tous les LSR du backbone. Là encore, ce n'est pas la solution que nous allons retenir. La solution idéale devrait être aussi automatique que possible, permettre au backbone d'être aussi simple que possible et ne demander que le moins de configuration possible pour les routeurs du provider. Cette solution, utilisée en pratique est le Provider-Provisioned MPLS VPN.



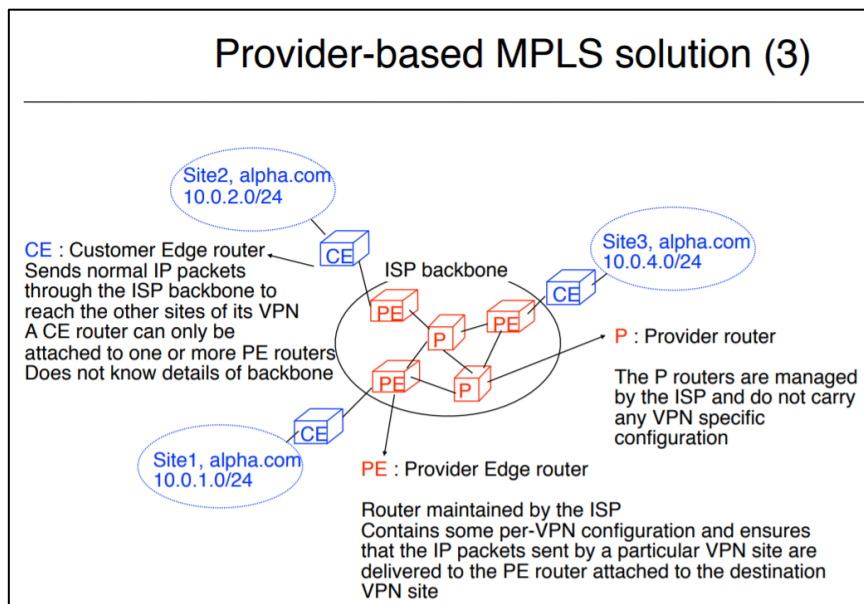
9.2 Provider-Provisioned MPLS VPN :

Cette solution consiste en un mixte de MPLS et de BGP. Le principe de base consiste à avoir trois types de routeurs. Les CE (Customer Edge) qui connaissent les routes accessibles via les autres CE mais n'ont aucune information concernant la structure interne du backbone. Les PE (Provider Edge), qui sont directement reliés à un ou plusieurs CE, vont posséder plusieurs tables de forwarding, une table par VPN qui y est attaché et une table pour la topologie interne du backbone. Ceci est dû à la possibilité pour plusieurs VPN d'utiliser les mêmes adresses virtuelles. Pour résoudre ce problème, les routeurs PE ont une table différente pour chaque VPN et vont utiliser un Road-distinguisher (expliqué plus loin) pour savoir quelle table utiliser. C'est le PE qui va s'assurer que les paquets provenant d'un VPN via son CE attaché arrivent bien (en traversant le backbone) à l'autre PE attaché à l'autre site du même VPN (destination).

Finalement, les routeurs P forment le backbone et n'ont aucune information concernant les VPN, uniquement concernant la topologie du backbone lui-même. Par exemple, sur l'image ci-dessous, les routeurs CE bleus connaissent les routes accessibles via les autres CE bleus mais rien de la topologie du backbone. Les PE ont une table propre au VPN bleu, une table propre au VPN vert et une table pour la topologie du backbone. Et les routeurs P n'ont que la table de routing du backbone.

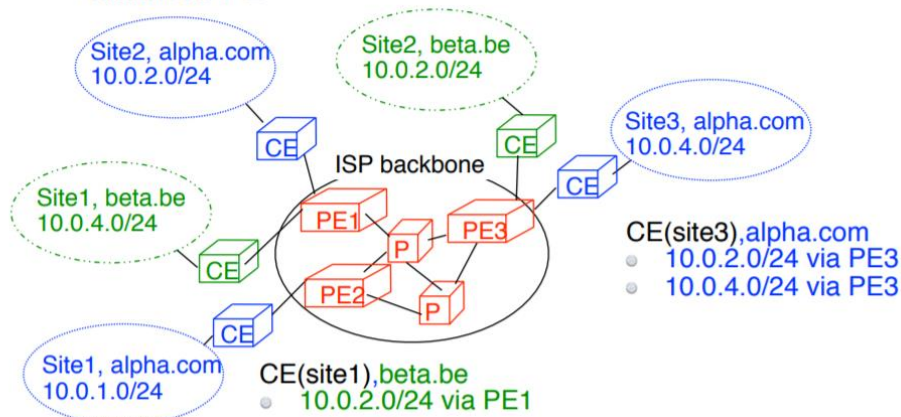
Voilà pour la configuration des routeurs, maintenant voyons comment le protocole fonctionne en pratique. Un CE appartenant à un VPN va advertise ses routes en eBGP au PE auquel il est attaché. Les PE vont être en iBGP full-mesh (on suppose que le network n'est pas énorme) et vont donc partager les nouvelles routes apprises pour que tous les PE soient au courant. Mais il reste le problème des adresses qui peuvent potentiellement être identiques entre différents VPN. Ce problème est réglé par une solution astucieuse ; le CE plutôt que d'advertise ses préfixes (préfixes d'adresses ipv4 ou ipv6) normalement, il va advertise des préfixes VPN-IP qui consistent en réalité à coller un identifiant suivi du préfixe qui doit être advertise. Cet identifiant placé au début de l'adresse, appelé Road-Distinguisher (RD) identifie de façon unique un site connecté à un CE (par exemple RD = ID du VPN suivi du ID du site). Les routeurs PE recevant ces routes comprennent cet encodage et vont utiliser le RD pour savoir à quel VPN l'adresse qui a été advertise fait référence (et donc dans quelle table mettre cette nouvelle route). A noter que seulement certains des PE doivent ajouter ces routes (les PE qui sont connectés au VPN en question) pour cela, des bgp communities sont utilisées (route target). Par exemple, dans la configuration de type « hub and spoke » où les paquets depuis spoke doivent passer par hub, il faut deux route target (communities) différentes, l'une représente « from spoke » et l'autre représentant « from hub ». Enfin, une combinaison avec du MPLS est utilisée en pratique avec deux niveaux de Labels. Un premier Label push par le PE d'entrée qui est utilisé pour joindre le PE de sortie (distribué par LDP ou RSVP-TE) et un deuxième Label (distribué via BGP) qui, une fois arrivé au PE de sortie, indique quel VPN est concerné (car le PE de sortie pourrait être connecté à plusieurs VPN utilisant les mêmes adresses virtuelles). Les routeurs du backbone doivent alors avoir la table de routing (OSFP) ainsi qu'une LFIB pour les tunnels MPLS.

Globalement, cette solution provider-provisioned, comparée à une solution customer-provisioned (telle que détaillée dans les principes généraux 9.1), offre une plus grande facilité de configuration (automatisme) mais requiert des communications ibgp et idéalement le support de MPLS sur les routeurs du backbone.



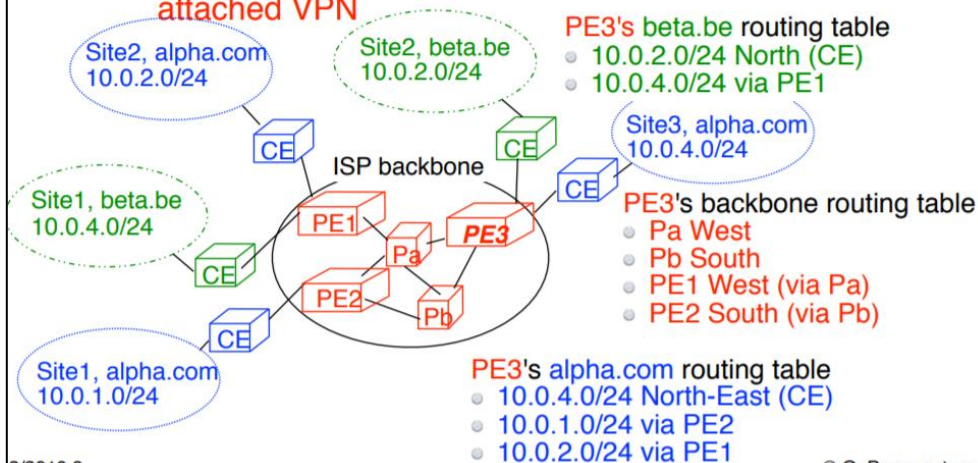
- **Principle**

- Each CE router contains one routing table with the routes belonging to its VPN
 - CE does not know anything about ISP besides its attached PE



- **Principle**

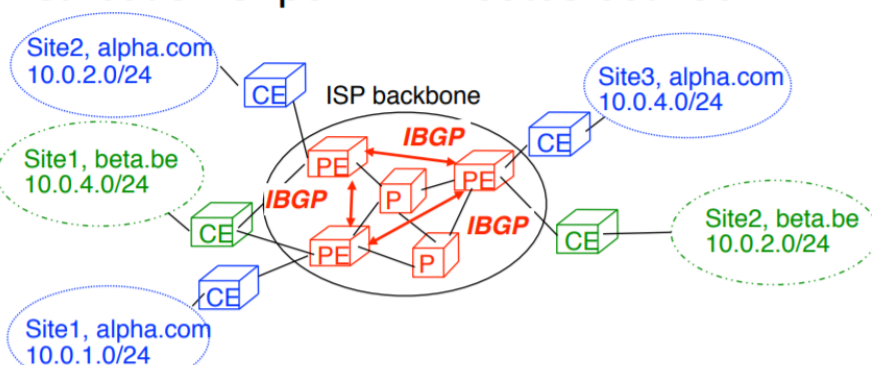
- Each PE router maintains several routing tables
 - standard routing table
 - one **VPN Routing and Forwarding table (VRF)** per attached VPN



3/2010.8

© O. Bonaventure 2

- **Distribution of per VPN routes between PE**



- **Principle**

- iBGP full mesh between PE routers
 - P routers do not need to run iBGP since they do not maintain per-VPN routes
- iBGP sessions are used to redistribute the routes learned from CE routers to distant PE routers

10.8

© O. Bonaventure 20

Solving the forwarding problem

- How to forward the packets from each VPN through the provider's backbone ?
 - sending pure IP packets is not possible
 - P routers cannot know VPN-specific routes
 - different VPNs use the same RFC1918 address space
- Principle of the solution
 - CE routers send normal IP packets
 - CE routers remain as simple as possible
 - PE routers maintain **several** routing tables
 - **one routing table per VPN attached to PE router**
 - **one routing table for the ISP backbone**
 - PE encapsulate VPN packets
 - Common solution is to encapsulate with MPLS
 - Some ISPs are using GRE, L2TP or IPSec

10. Segment Routing

RSVP-TE qui a été vu au point 6.2 a tendance à ne pas bien scale à grande échelle car il faut échanger des paquets PATH et RESVP entre tous les routeurs du chemin et ce de façon régulière (soft state). Ce qui signifie un grand nombre d'échanges et est globalement très mauvais pour un gros network avec beaucoup de routeurs. Par ailleurs, LDP a aussi plusieurs défauts notamment le fait que, contrairement à RSVP-TE, il ne peut pas imposer un chemin particulier et suit simplement le chemin le plus court. De plus, il nécessite une synchronisation avec les autres protocoles de routing (OSPF et iBGP) qui doivent être finis (établis et convergés) avant de pouvoir run LDP (sinon des Labels pourraient être perdus dans des blackholes). Globalement, MPLS nécessite que des Labels soient distribués et les deux méthodes de distribution de ces Labels sont imparfaites. Segment routing permet une alternative à MPLS pour permettre à un paquet de suivre un chemin particulier sans nécessiter de run LDP ou RSVP-TE au préalable. L'idée est la suivante : Chaque routeur a un Label unique (appelé node routing), par exemple son routeur ID. Ce Label unique va être advertise via OSPF en l'ajoutant dans les Link State Packets. Ces Labels très facilement distribués vont ensuite être utilisés pour faire du Segment Routing (ou adjacency routing). Il suffit de placer une stack de Label dans le paquet et ceux-ci vont permettre d'acheminer le paquet en le dirigeant vers chacun des routeurs identifiés par les Labels. Les routeurs généraux (core) devront simplement avoir la table OSPF et une petite LFIB tandis que le routeur à l'origine du segment routing devra avoir une table OSPF, une petite table LFIB et connaître le chemin qu'il souhaite faire suivre au paquet. A noter que le chemin choisi pour aller d'un routeur à un autre (d'un Label à un autre) suit automatiquement le chemin le plus court, certains chemins spécifiques ne peuvent donc pas être forcés (comme c'était le cas via MPLS distribué par LDP, alors que tout chemin était possible via MPLS distribué par RSVP-TE). Sur l'image suivante, par exemple, il n'est pas possible

de forcer le chemin vert sur l'image suivante avec du segment routing (SR). Tandis que le chemin rouge peut être suivi en plaçant la stack de Labels : R1|R3|R7. Pour le chemin bleu, une astuce est possible en définissant deux chemins vers R8 dans la table de R2 et en leur définissant tous les deux le même coût. Il suffit alors d'utiliser la stack de Labels : R8|R9|R7. Cependant, le chemin vert, comme dit précédemment, n'est pas réalisable, en effet, arrivé à R3, si on plaçait le Label R5, le routeur R3 enverrait le paquet vers R7 pour atteindre R5 car cela représente le chemin le plus court. Une solution pour permettre de suivre n'importe quel chemin est d'utiliser du adjacency routing. Il s'agit du même principe que le segment routing sauf que dans la stack, on précise les Labels et les interfaces de sortie. Par exemple, pour le chemin rouge, on peut faire une stack : R1 via interface N-E, R3 via interface E, R5 via interface S, R7 via interface E. Cela permet donc de forcer n'importe quel chemin (mais cela demande de connaître parfaitement la topologie avec les différentes interfaces des autres routeurs). Le segment routing (ou adjacency routing si nécessaire) est très utile dans le cas où aucun loop-free alternate (voir section 8.2) n'est présent. En effet, nous avons parlé du fait qu'il existe toujours un loop-free alternate distant mais rediriger le trafic vers celui-ci n'était pas possible car le premier voisin à recevoir les paquets allait rediriger le paquet vers nous et donc créer une boucle. La solution est alors de rediriger le trafic vers le loop-free alternate distant en utilisant du segment (ou adjacency) routing afin de forcer cette portion du chemin des paquets (jusqu'au loop-free alternate distant).

- Which paths require the utilisation of adjacency segments ?
 - How many labels do you need for each path ?

